

SmolVLA 추론 코드 해설

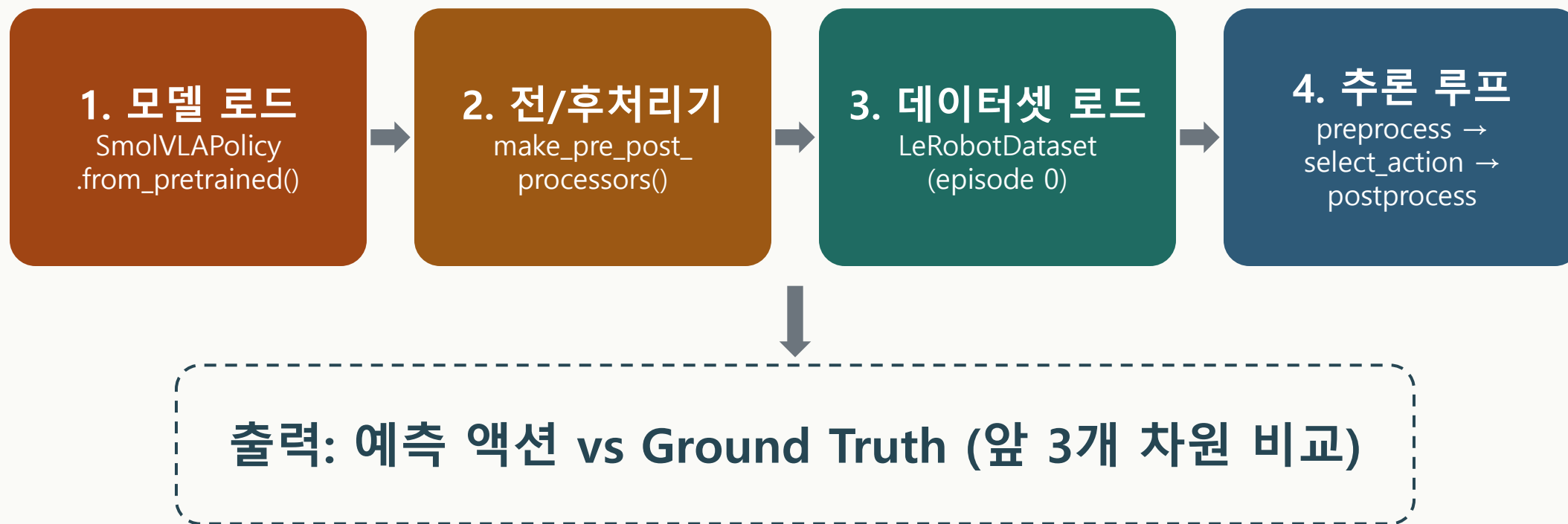
04b_inference_smolvla.py

로봇 없이 LeRobot 데이터셋 프레임으로 SmolVLA 추론하기

이 스크립트가 하는 일

- 실제 로봇/시뮬레이터 **없이**, 공개 LeRobot 데이터셋의 프레임을 모델에 입력
- **SmolVLA**가 액션을 예측 → 데이터셋의 실제 액션(GT)과 비교
- 모델이 "말 + 영상 → 행동(Action)"을 어떻게 다루는지 확인하는 학습용 코드

전체 파이프라인



Step 1: 모델 + 전/후처리기 로드

```
policy = SmolVLAPolicy.from_pretrained(  
    "lerobot/smolvla_base")  
policy.eval(); policy.to(device)
```

```
preprocessor, postprocessor = \  
make_pre_post_processors(  
    policy_cfg=policy,  
    pretrained_path=MODEL_ID,  
    preprocessor_overrides=...)
```

- **from_pretrained**: HuggingFace에서 사전학습 SmolVLA 가중치 다운로드
- **preprocessor**: 정규화, 토큰화, 디바이스 이동을 한 번에
- **postprocessor**: 역정규화 + CPU 이동
- 04 버전에서 직접 찐던 normalize 코드를 **자동화**
- `policy.config.input_features`로 모델이 기대하는 키 확인 가능

Step 2: 데이터셋 로드

```
# 공개 데이터셋 (so100 로봇 pick&place)
DATASET_ID = "lerobot/svla_so100_pickplace"

dataset = LeRobotDataset(
    DATASET_ID, episodes=[0])

frame = dataset[i] # dict
```

- **LeRobotDataset**: HuggingFace 데이터셋 자동 다운로드 + 캐시
- **episodes=[0]**: 0번 에피소드만 로드 (메모리 절약)
- 각 프레임은 dict: 카메라 영상, 로봇 상태, action, task 문자열 등 포함
- 실제 로봇 없이 **녹화된 관측치**로 추론을 시뮬레이트

frame = dataset[i] 내부 구조

 **frame = dataset[0]** # dict 형태

"observation.images.top": Tensor (3, 480, 640)

"observation.images.wrist": Tensor (3, 480, 640)

"observation.state": Tensor (6,)

"action": Tensor (6,)

"task": str

"timestamp": float

"frame_index": int

"episode_index": int

관측(Observation)

- images.* → 카메라 영상 (RGB, HxW)
- state → 로봇 관절 6축 현재 위치

행동(Action) — GT

- 6D 벡터: 6축 관절 목표 위치
- 모델 예측과 비교할 정답값

언어(Task)

- 자연어 명령 (예: "pick up the cube...")
- SmolVLA의 'L' 입력으로 사용

메타데이터

- frame_index, episode_index, timestamp
- 프레임의 위치/시간 정보

실제 데이터 샘플 (frame=42 예시)

"observation.image
s.top"



책상 위 RGB 이미지
shape: (3, 480, 640)

"observation.image
s.wrist"



그리퍼 시점 RGB
shape: (3, 480, 640)

"task" (str)

**"pick up the cube
and place it in the box"**

→ 모델에 자연어 명령으로 들어감

"observation.state" Tensor (6,) — 현재 6축 관절 각도(°)

tensor([12.3, -45.7, 78.2, 10.1, -3.4, 0.0])

shoulder · elbow · wrist · gripper · ... 순서

"action" (6,)

다음 스텝 목표 각도 (GT)

[12.5, -45.2,
78.8, 10.5,
-3.1, 0.0]

"frame_index": 42

"episode_index": 0

"timestamp": 1.4 sec

state(현재 위치) → 모델 → action(다음 위치) — state와 action의 차이가 "움직임 한 스텝"

Step 3: 추론 루프 — 4가지 핵심 처리

1

배치 차원 추가

각 텐서 필드에
unsqueeze(0)
으로 배치 차원 추가

2

이미지 키 매핑

데이터셋 키
(top/wrist) →
모델 키(camera1/2/3)
부족분은 0으로 채움

3

task 문자열

VLA의 'L' (Language)
자연어 명령 필수
예: "pick up the
cube..."

4

전.후처리 + 추론

preprocessor →
select_action() →
postprocessor

이 4단계가 한 프레임당 1번씩, NUM_FRAMES만큼 반복됩니다.

비유 — 운전학원에서 배우듯이

옛날 방식

강사가 말로 설명

→ 규칙 기반 (if/else 코드)

AI 방식

강사 운전을 1000시간 녹화

→ 데이터로 학습

사람 시범 → 데이터 → AI 학습 → 로봇 동작

로봇팔도 똑같다 — 사람 시범을 30fps로 녹화해 학습

우리가 쓸 데이터셋

`lerobot/svla_so100_pickplace`

누가
사람이 SO-100 로봇팔을 조작

무엇을
큐브 집어 박스에 넣기

얼마나
약 15초 시범, 여러 번 반복

어떻게
30fps 녹화 (초당 30번)

한 번의 시범 = 약 454 프레임 ($15.1\text{초} \times 30\text{fps}$)

한 프레임 = AI에게 주는 한 장의 카드

FRAME 0

1/30

초마다 한 카드

1초 = 30 카드

15초 = 454 카드

위 카메라 480×640 RGB

손목 카메라 480×640 RGB

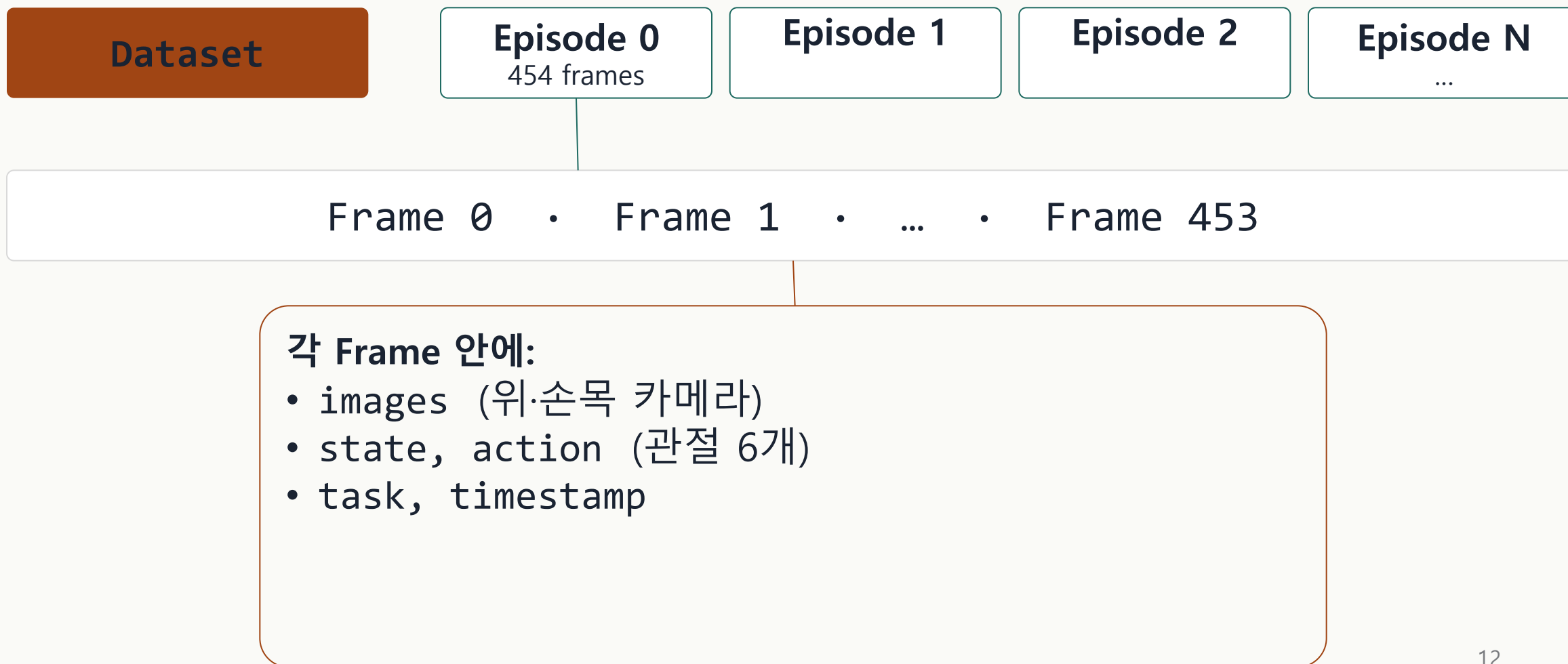
현재 관절 (state) 6개 숫자

다음 동작 (action) 6개 숫자

작업 명령 (task) "Pick up the cube..."

시각 (timestamp) 0.0초

데이터셋 전체 구조



state vs action — 학습의 핵심

observation.state

현재 관절 각도

[0.35, 176.84, 165.06,
72.07, 82.97, 0.35]

action

다음에 갈 각도 (정답)

[0.62, 177.19, 164.62,
72.33, 82.53, 0.11]

AI의 학습 목표: (사진 + state + 명령) → action 맞추기

시간이 흐르면 action은 어떻게 변할까?

Frame 0~30: action 변화

Frame 0: [0.615, 177.187, ...]

Frame 6: [0.615, 177.187, ...]

Frame 12: [0.615, 177.187, ...]

Frame 15: [0.703, 177.187, ...]

14프레임 만에 베이스 변화!

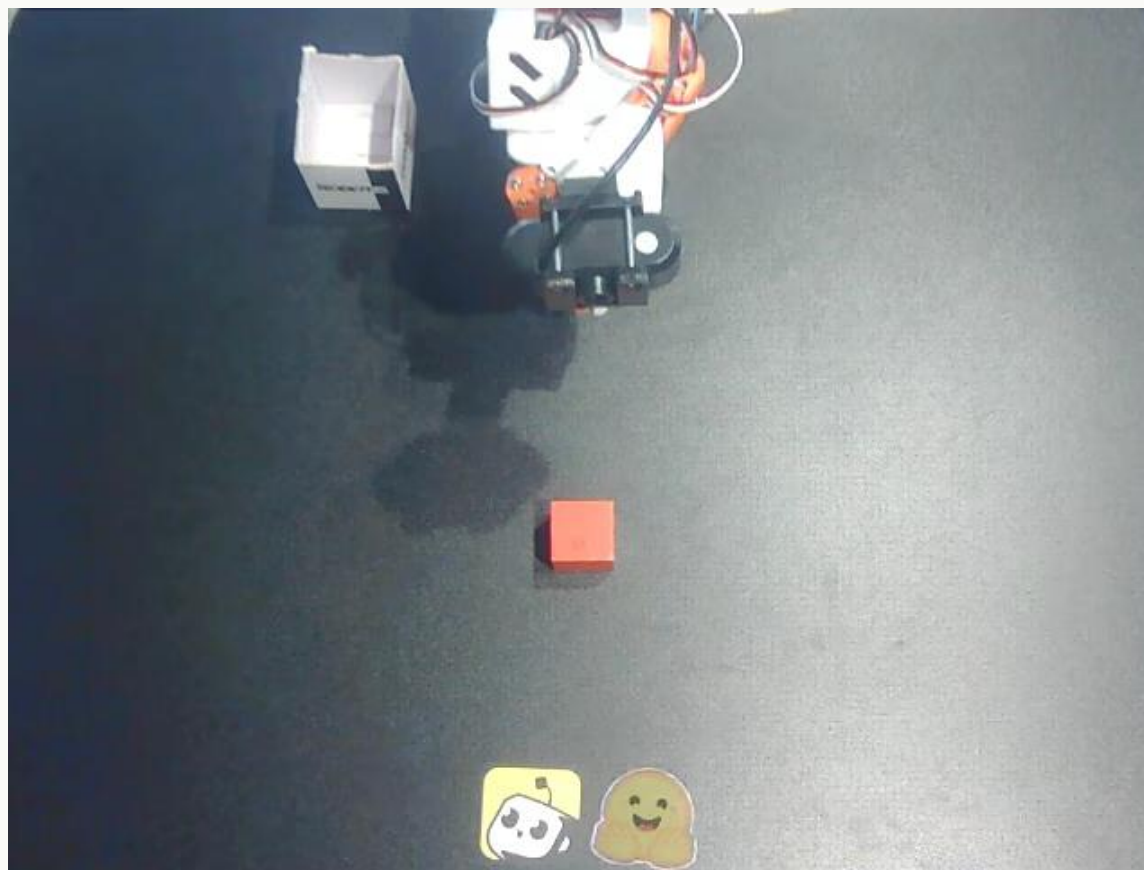
관찰

- 처음 0.5초 정지
- 15F~ 베이스 시작
- 다른 관절 정지

데이터 = 시간의 기록

lerobot/svla_so100_pickplace

C:\Users\dongk\cache\huggingface\lerobot\lerobot\svla_so100_pickplace



3-1. 배치 차원 추가

```
# dataset[i]는 단일 샘플 dict
frame = dataset[i]
```

```
batch = {}
for k, v in frame.items():
    if isinstance(v, torch.Tensor):
        batch[k] = v.unsqueeze(0)
```

왜 필요할까?

- 모델은 항상 **배치 단위**로 입력을 받음
- 한 프레임만 추론해도 첫 차원에 1을 추가

예시: 카메라 이미지

shape: (3, 480, 640)

↓ **unsqueeze(0)**

shape: (1, 3, 480, 640)

3-2. 이미지 키 매핑

```
# 모델이 기대 vs 데이터셋이 제공
model_img_keys = sorted([...])
dataset_img_keys = sorted([...])

for idx, m_key in enumerate(model_img_keys):
    if idx < len(dataset_img_keys):
        batch[m_key] = batch[
            dataset_img_keys[idx]]
    else:
        batch[m_key] = torch.zeros(...)
```

왜 필요할까?

- 모델/데이터셋이 카메라 키 이름이 다름
- 카메라 수가 부족하면 0으로 채움

예시

```
# 데이터셋: 2개
obs.images.top, .wrist
```

↓ 매핑

```
# 모델: 3개 필요
- camera1 ← top
- camera2 ← wrist
- camera3 ← zeros
```

3-3. task 문자열 — 자연어 명령

```
# 데이터셋에 task 있으면 그대로 사용
if "task" in frame:
    batch["task"] = [frame["task"]]
else:
    batch["task"] = [
        "pick up the cube and
        place it in the box"]
```

왜 필요할까?

- **VLA** = Vision + **Language** + Action
- 같은 영상이라도 명령이 다르면 **다른 행동**

예시

- 같은 책상 위 영상 +
 - "pick up the **cube**" → 큐브로 이동
 - "pick up the **ball**" → 공으로 이동
- 리스트로 감싸는 이유: 배치 형태

3-4. 전·후처리 + 추론

1) 전처리: 정규화 + GPU 이동

```
processed = preprocessor(batch)
```

2) 추론: 액션 한 스텝 예측

```
action = policy.select_action(processed)
```

3) 후처리: 역정규화 + CPU 이동

```
action = postprocessor(action)
```

```
pred = action.squeeze().cpu().numpy()
```

각 단계의 역할

preprocessor

- 이미지 [0,255] → [-1,1] 정규화
- task 문자열 → 토큰 ID
- 모든 텐서 → GPU

select_action

- 50-step chunk에서 다음 1스텝 반환

postprocessor

- 액션 [-1,1] → 실제 로봇 단위
- GPU → CPU

알아둘 점 & 다음 단계

- **Action Chunk**: SmolVLA는 한 번의 forward로 **50-step** 액션을 예측
 - `select_action()`은 chunk에서 다음 한 스텝씩 꺼내 반환
- **예측 $\neq$ GT**는 정상 — 사전학습 모델을 그대로 사용
 - 특정 데이터셋에 맞추려면 **fine-tuning** 필요
- **VLA = Vision + Language + Action**
 - 영상(camera) + 언어(task) + 상태(state) $\rightarrow$ 6D 액션

프로그램 출력 화면

```
Running inference on 10 frames...
```

Frame	Predicted (first 3 dims)	Ground Truth
0	[-0.415, 0.972, 0.080]	[0.615, 177.188, 164.619]
1	[-0.219, 0.954, -0.427]	[0.615, 177.188, 164.619]
2	[-0.585, 1.222, -0.586]	[0.615, 177.188, 164.619]
3	[-0.463, 1.166, -0.690]	[0.615, 177.188, 164.619]
4	[-0.309, 1.170, -0.812]	[0.615, 177.188, 164.619]
5	[0.055, 1.052, -0.961]	[0.615, 177.188, 164.619]
6	[0.218, 1.052, -0.583]	[0.615, 177.188, 164.619]
7	[0.193, 1.133, -0.222]	[0.615, 177.188, 164.619]
8	[0.352, 1.220, 0.375]	[0.615, 177.188, 164.619]
9	[0.441, 1.229, 0.609]	[0.615, 177.188, 164.619]

```
Inference complete!
```

```
Note:
```

- SmolVLA는 한 번의 forward로 50-step action chunk를 예측합니다. select_action()은 chunk에서 다음 한 스텝씩 꺼내서 반환합니다.
- 예측과 GT가 항상 일치하지는 않습니다. 사전학습 모델을 그대로 쓴 것이고, fine-tune을 하면 특정 데이터셋에 더 잘 맞춰집니다.

숙제

04d_prompt_smolvla.py 프로그램 분석하기

```
== prompt별 첫 액션 비교 (앞 3 dim) | baseline 대비 L2 ==  
-----  
prompt | action[:3] | L2 vs base  
-----  
pick up the cube and place it in the box | [-0.183, 0.155, 0.127] | 0.0000  
pick up the cube and put it on the table | [-0.440, 0.483, 0.025] | 1.1645  
move the arm to the left | [-0.643, 0.355, 0.929] | 2.3765  
stay still and do nothing | [ 0.689, 1.296, 0.710] | 2.2136  
make a sandwich | [ 0.229, 0.522, -0.011] | 0.7787  
(empty) | [ 0.731, -0.135, 1.175] | 1.7360  
-----  
  
== 차원별 prompt-induced 변동성 (std across prompts) ==  
dim 0: 0.5286  
dim 1: 0.4398  
dim 2: 0.4671  
dim 3: 0.3094  
dim 4: 0.8569  
dim 5: 0.4318  
  
== Paraphrase vs Distractor 비교 ==  
PARAPHRASES (같은 의미):  
- 'pick up the cube and place it in the box'  
- 'put the cube into the box'  
- 'place the cube inside the box'  
- 'grab the cube and drop it in the box'  
DISTRACTORS (다른 의미):  
- 'move the arm to the left'  
- 'stay still and do nothing'  
- 'make a sandwich'  
  
intra (paraphrase ↔ paraphrase) 평균 L2: 2.2425  
inter (paraphrase ↔ distractor) 평균 L2: 1.8652  
ratio (inter / intra) : 0.83x  
  
→ inter ≤ intra: 의미 구분이 약하거나, prompt들이 사전학습 도메인에서  
멀어 신호 자체가 노이즈에 가까운 상태일 수 있음 → fine-tune의 동기.
```